

Cụm 5 - Fundamental Architecture Styles

Mục lục

5.1 Tổng quan Cụm 5: Fundamental Architecture Styles	4
Nếu bạn đến từ Data Science	4
Style vs Pattern	4
9 Architecture Styles trong khoá	4
Cụm 5 (Fundamental - monolithic-friendly)	4
Cụm 6 (Distributed)	4
Bài trong cụm	5
5.2: Monolithic vs Distributed	5
5.3: Layered Architecture	5
5.4: Pipeline Architecture	5
5.5: Microkernel Architecture	5
Tại sao chỉ 3 style + Monolithic baseline ở Cụm 5?	5
Cách đọc cụm	5
5.2 Monolithic vs Distributed	6
Nếu bạn đến từ Data Science	6
Định nghĩa	6
Monolithic Architecture	6
Distributed Architecture	7
Monolithic là default	7
Lợi ích monolithic	7
Khi nào monolithic không đủ	7
8 Distributed Computing Fallacies	7
Hidden costs của Distributed	7
Cost 1: Latency và bandwidth	7
Cost 2: Partial failure	9
Cost 3: Eventual consistency	9
Cost 4: Distributed tracing	9
Cost 5: Operational complexity	9
Cost 6: Testing	9
Cost 7: Network security	9
Trade-off table	9
Modular Monolith, middle ground	10
Strangler Fig pattern	10
Sai lầm thường gặp	10
Sai lầm 1: Microservices vì hype	10
Sai lầm 2: Distributed monolith	10

Sai lầm 3: Quên 8 fallacies	10
Sai lầm 4: Không có observability từ ngày 1	11
Sai lầm 5: Premature optimization	11
Tóm tắt	11
5.3 Layered Architecture	12
Nếu bạn đến từ Data Science	12
Topology	12
Closed vs Open layers	12
Closed layer (default)	12
Open layer	12
Sinkhole Anti-pattern	12
Khi dùng Layered	14
Trade-off với QA	15
Implementation trong code	15
Java/Spring example	15
Python/Flask example	16
Variants	16
Layered + Hexagonal	16
Onion / Clean Architecture	16
Sai lầm thường gặp	16
Sai lầm 1: Skip layers ad-hoc	16
Sai lầm 2: Layered cho complex domain	16
Sai lầm 3: Database-first design	17
Sai lầm 4: Repository pattern abuse	17
Tóm tắt	17
5.4 Pipeline Architecture	18
Câu hỏi mở đầu	18
Topology cốt lõi	18
Nếu bạn đến từ Data Science	18
Bốn loại filter	19
1. Producer	19
2. Transformer	19
3. Tester	19
4. Consumer hoặc sink	20
Unix Philosophy và vì sao nó vẫn đúng	20
Ví dụ 1: Training pipeline	20
Ví dụ 2: Batch inference pipeline	21
Ví dụ 3: Streaming feature pipeline	21
Cách implement pipeline	22
In-process pipeline	22
Orchestrated batch pipeline	22
Distributed streaming pipeline	22
Trade-off với Quality Attributes	23
Khi nào dùng Pipeline Architecture?	23
Phù hợp	23
Không phù hợp	23
Design rules cho pipeline tốt	24

Rule 1: Contract giữa filters phải rõ	24
Rule 2: Filter càng stateless càng tốt	24
Rule 3: Side effect phải nằm ở sink rõ ràng	24
Rule 4: Error path cũng là một pipeline	24
Rule 5: Mỗi step cần metric riêng	24
Sai lầm thường gặp	25
Sai lầm 1: Notebook là pipeline production	25
Sai lầm 2: God filter	25
Sai lầm 3: Không lưu intermediate output	25
Sai lầm 4: Không version schema và model	25
Sai lầm 5: Chọn streaming khi batch đủ	25
Self-check	26
Tóm tắt	26
5.5 Microkernel Architecture	27
Nếu bạn đến từ Data Science	27
Topology cốt lõi	27
Lợi ích vs Layered	27
Examples thực tế	28
VS Code	28
Eclipse IDE	28
Chrome / Firefox	28
WordPress	28
CMS / E-commerce platforms	28
DAW (Digital Audio Workstation)	28
Plug-in Contract Design	28
Loại 1: Event/Hook system	28
Loại 2: Service registration	29
Loại 3: Command pattern	29
Loại 4: Contribution points (Eclipse model)	30
Plug-in Lifecycle	30
Sandboxing (cho third-party plug-ins)	30
Process isolation	30
Sandboxed runtime	30
Permission system	30
Trade-off với QA	30
Khi dùng Microkernel	31
Variants	31
Microkernel + Distributed plug-ins	31
Microkernel + Module Federation (Frontend)	31
Sai lầm thường gặp	32
Sai lầm 1: Core quá lớn	32
Sai lầm 2: API không backward compatible	32
Sai lầm 3: Plug-in coupling lẫn nhau	32
Sai lầm 4: Performance overhead bị bỏ qua	32
Sai lầm 5: Security model yếu	32
Tóm tắt	32
Tổng kết Cụm 5	32

5.1 Tổng quan Cụm 5: Fundamental Architecture Styles

Tóm tắt một dòng: Architecture Style là “shape” tổng thể của hệ. Cụm này dạy 4 style fundamental (monolithic-friendly): Monolithic baseline, Layered, Pipeline, Microkernel, và quyết định cơ bản nhất: monolithic hay distributed.

Nếu bạn đến từ Data Science

Trong Cụm 5, bài bạn nên đọc kỹ nhất là Pipeline Architecture. Đây là style gần nhất với ETL, feature engineering, training pipeline và batch scoring. Layered giúp bạn hiểu cách tách API, business logic và persistence. Microkernel giúp bạn hiểu cách thêm plug-in model, metric hoặc feature transform mà không sửa core.

Đừng xem architecture styles như danh sách tên để thuộc. Hãy xem chúng như các hình dạng tổ chức hệ thống. Một notebook lớn là một hình dạng. Một Airflow DAG là một hình dạng khác. Một model serving API với feature store lại là hình dạng khác nữa.

Style vs Pattern

Phân biệt:

- **Architecture Style:** shape tổng thể của hệ. Vd: “monolithic”, “microservices”, “event-driven”. Quyết định một lần cho cả hệ (thường).
- **Architecture Pattern:** solution cụ thể cho một bài toán con. Vd: “Saga”, “Circuit Breaker”, “CQRS”, “Strangler Fig”. Một hệ có thể dùng nhiều pattern.

Trong khoá này, Cụm 5 + 6 đi qua 9 styles. Pattern được nhắc lúc relevant nhưng không là focus chính.

9 Architecture Styles trong khoá

Cụm 5 (Fundamental - monolithic-friendly)

Style	Topology	Khi dùng
Layered	Tầng UI/Business/Data	Default cho monolith, app đơn giản
Pipeline	Filter nối tiếp	Data processing, ETL, compiler
Microkernel	Core + plug-ins	IDE, browser, CMS, product có extensibility

Plus Monolithic vs Distributed comparison (Bài 5.2).

Cụm 6 (Distributed)

Style	Topology	Khi dùng
Service-based	4-12 coarse services + DB shared	Enterprise medium, ít risk hơn microservices
Microservices	Many fine-grained services + DB per service	Large scale, multi-team
Event-Driven	Async message broker	Real-time, high throughput, decoupling

Style	Topology	Khi dùng
Space-Based	In-memory grid + DB async sync	Extreme load (Black Friday)

Bài trong cụm

5.2: Monolithic vs Distributed

Trade-off cơ bản nhất. Vì sao monolithic là *default*, khi nào lên distributed. “Distributed Computing Fallacies”, 8 sai lầm phổ biến. Cost của distributed: latency, partial failure, debugging.

5.3: Layered Architecture

Style phổ biến nhất historical. UI / Business / Data layer. Closed vs Open layer. Strengths: simple, well-understood. Weaknesses: hard to scale parts, “sinkhole anti-pattern”.

5.4: Pipeline Architecture

Filter + Pipe. Sequential transformation. Unix philosophy. Strengths: composable, easy to add filter. Use cases: ETL, compiler, image processing.

5.5: Microkernel Architecture

Core (kernel) + Plug-ins. OCP at scale. Examples: VS Code, Eclipse, Chrome, WordPress. Strengths: extensibility, third-party ecosystem. Weaknesses: plugin compatibility, performance overhead.

Tại sao chỉ 3 style + Monolithic baseline ở Cụm 5?

Cụm 5 cover **fundamental styles**, những style mà toàn hệ chạy trong 1 process (typical monolith), hoặc 1 process với plug-ins. Cụm 6 cover **distributed styles**, nhiều process/network.

Reason tách: monolithic thinking và distributed thinking khác nhau cơ bản. Distributed thêm cost mới (network, partial failure, eventual consistency) cần cách reasoning mới.

Cách đọc cụm

Tuần tự 5.2 □ 5.3 □ 5.4 □ 5.5. Hoặc đọc 5.2 first và skip nếu đã biết, sau đó chọn style đang relevant với dự án mình.

Mỗi bài có pattern chung:

1. **Topology**: vẽ diagram cốt lõi.
2. **Khi nào dùng**.
3. **Khi nào không**.
4. **Trade-off** với QA (Cụm 4): style này tối ưu QA gì, hy sinh QA gì.
5. **Real-world examples**.

Vào [Bài 5.2: Monolithic vs Distributed](#) để bắt đầu.

5.2 Monolithic vs Distributed

Tóm tắt một dòng: Monolithic = toàn hệ chạy 1 process; Distributed = nhiều process qua network. Monolithic luôn nên là default. Distributed *chỉ* khi có lý do mạnh, vì nó thêm 5x complexity và 5x operational cost.

Nếu bạn đến từ Data Science

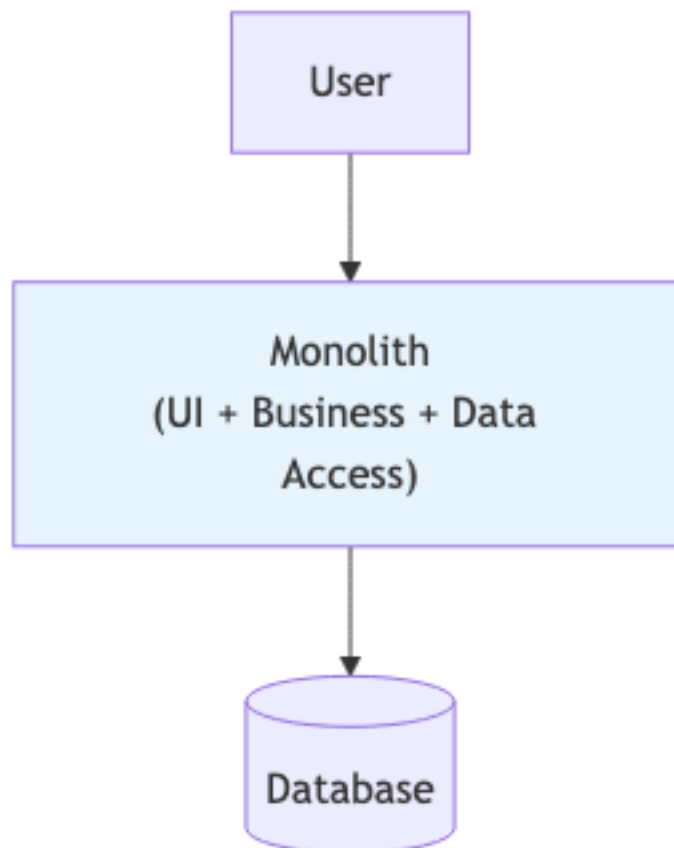
Hành trình production ML thường đi qua nhiều mức. Ban đầu là notebook. Sau đó là một script batch. Sau đó là một package có module rõ. Tiếp theo có thể là một service-based ML platform với feature service, training orchestrator, model registry và serving service. Không nên nhảy từ notebook thẳng sang microservices.

Monolith không phải xấu nếu nó modular. Một batch scoring job rõ ràng, có test, có monitoring, có rollback có thể tốt hơn nhiều so với 10 service nhỏ nhưng coupling chặt và không ai vận hành nổi.

Định nghĩa

Monolithic Architecture

Toàn bộ application code chạy trong 1 process duy nhất (hoặc 1 container/server, replicate cho HA).



Hình 1: Sơ đồ 12

Communication giữa các module = function call trong process. Fast (nanoseconds), reliable (no network).

Distributed Architecture

Application chia thành nhiều process độc lập, giao tiếp qua network.

Communication = HTTP/gRPC/message. Slow (milliseconds), unreliable (network failures).

Monolithic là default

Quy tắc thực hành: nếu bạn chưa có lý do *cực mạnh* để distributed, hãy monolithic.

Lợi ích monolithic

1. **Đơn giản phát triển.** 1 codebase, 1 deploy, 1 database. Onboard dev nhanh.
2. **Performance cao.** Function call < 1 microsecond. Distributed call > 1ms.
3. **Transaction dễ.** ACID trong 1 database. Distributed transaction (saga, 2PC) phức tạp.
4. **Debug dễ.** Stack trace đầy đủ. Distributed: distributed tracing required (Jaeger, Zipkin).
5. **Cost thấp.** 1 server + 1 DB = \$50/tháng. Distributed setup minimum = \$500/tháng.

Khi nào monolithic không đủ

Tín hiệu phải lên distributed:

- **Team scale:** > 50 engineers, tuần nào cũng merge conflict trên monolith.
- **Different scaling:** 1 module ăn 90% traffic, không thể scale riêng.
- **Different tech stack:** ML team cần Python, web team cần Node, blockchain cần Rust.
- **Independent deploy:** cần ship 10 lần/ngày từ team này mà không block team kia.
- **Fault isolation:** 1 module down không được vỡ toàn hệ.

Nếu *không* match ≥ 2 điểm trên $\square$ monolithic vẫn ổn.

8 Distributed Computing Fallacies

Peter Deutsch + James Gosling (Sun Microsystems, 1994) liệt kê 8 sai lầm developer mới hay tin về distributed system:

1. **The network is reliable,** Sai. Packet drops, switches fail, cable bị đào lên.
2. **Latency is zero,** Sai. Network call 1-100ms vs local call < 1us.
3. **Bandwidth is infinite,** Sai. Có limit, có cost.
4. **The network is secure,** Sai. MITM, sniffing nếu không TLS.
5. **Topology doesn't change,** Sai. Nodes lên xuống, IPs thay đổi.
6. **There is one administrator,** Sai. Multiple teams config, có conflict.
7. **Transport cost is zero,** Sai. Network cost cao, đặc biệt cross-region.
8. **The network is homogeneous,** Sai. Heterogeneous protocol, latency, OS.

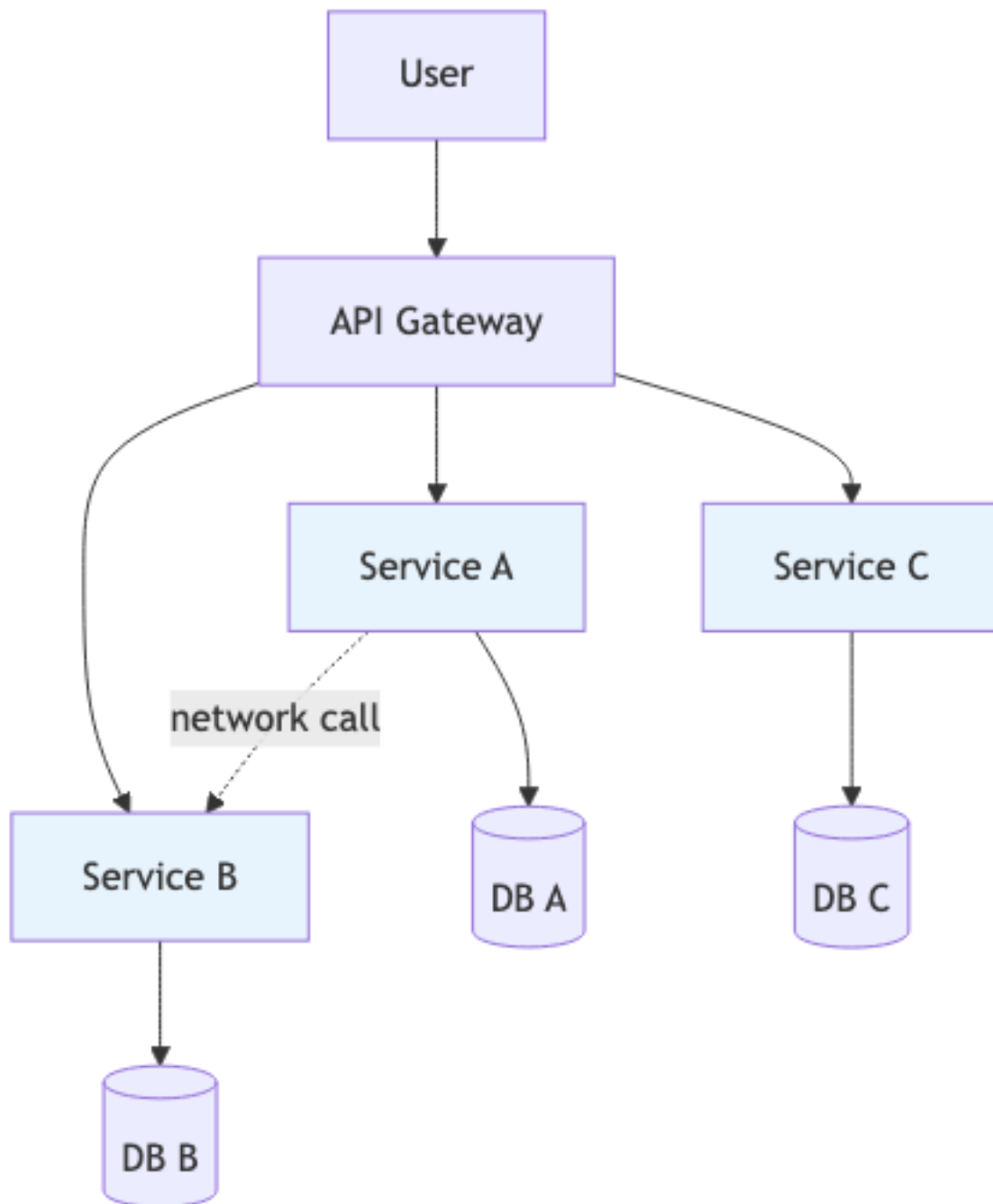
Mỗi fallacy = một class bug trong production. Distributed system architect phải handle cả 8.

Hidden costs của Distributed

Khi quyết định distributed, ngoài lợi ích visible, có cost ẩn:

Cost 1: Latency và bandwidth

Local call = nanoseconds. HTTP/gRPC call = 1-10ms in best case, 100ms+ across regions. Chain 5 services = 50ms+ latency added.



Hình 2: Sơ đồ 13

Cost 2: Partial failure

Monolith: hoặc tất cả lên, hoặc tất cả down. Distributed: service A down, B up, phải handle gracefully (circuit breaker, fallback).

Cost 3: Eventual consistency

Distributed transaction phức tạp. Đa số case dùng eventual consistency (saga, event-driven). Business team phải hiểu data có thể stale.

Cost 4: Distributed tracing

Bug trong distributed □ phải trace request qua N services. Cần Jaeger/Zipkin + correlation ID. Setup phức tạp.

Cost 5: Operational complexity

- 1 monolith: monitor 1 service, 1 deploy pipeline.
- 20 microservices: monitor 20 services, 20 pipelines, service mesh, secrets management...

Operational cost (DevOps, SRE) tăng 5-10x.

Cost 6: Testing

End-to-end test cần spin up multiple services. Slow, flaky. Cần test pyramid khác (more contract testing, less E2E).

Cost 7: Network security

Mỗi inter-service call cần TLS, auth (mTLS hoặc JWT). Cert rotation, secret management.

Trade-off table

QA	Monolithic	Distributed
Performance	High (in-process)	Medium-Low (network)
Scalability (uniform)	Vertical scale only	Horizontal scale per service
Scalability (varied)	Cannot scale parts	Excellent, scale only hot parts
Availability	All or nothing	Partial, fault isolated
Maintainability	Easy when small, hard when large	Harder per service but easier across team
Deployability	1 deploy = all changes	Independent deploys
Testability	Easy (in-process)	Hard (E2E + contract test)
Observability	Easy (1 log stream)	Hard (distributed tracing required)
Time-to-market (early)	Fast	Slow (setup overhead)
Time-to-market (mature, large team)	Slow (merge hell)	Fast (parallel team work)
Cost (small)	Low (\$50-500/mo)	High (\$500-5000/mo)

QA	Monolithic	Distributed
Cost (large)	High (1 huge server)	Optimized (scale only needed parts)

Insight: trade-off đảo theo scale. Monolithic tốt ở scale nhỏ, distributed tốt ở scale lớn, crossover khoảng 20-50 engineers + 100k+ users.

Modular Monolith, middle ground

Trước khi nhảy lên distributed, consider modular monolith:

- 1 deployment (1 process, 1 DB).
- *Nhưng* code organized thành strict modules với boundary cứng (mỗi module có interface công khai, không import internal lẫn nhau).
- Build tool enforce: module A không import từ `module_b.internal.*`.

Lợi ích:

- Có lợi ích của monolithic (simple deploy, in-process call).
- Có lợi ích của modularity (vertical slicing, team ownership).
- Migrate sang microservices dễ sau này (mỗi module $\square$ service).

Tools: Java module-info.java, Spring Modulith, .NET assemblies, Python package structure with strict imports.

Heuristic: hầu hết “we need microservices” thực sự “we need modular monolith”. Cố modular monolith trước, distributed khi pain thực sự.

Strangler Fig pattern

Khi đã có monolith và cần lên distributed, đừng rewrite from scratch (high risk). Dùng Strangler Fig:

1. Build API gateway/proxy in front of monolith.
2. Cứ feature mới, build microservice mới, route traffic qua gateway.
3. Dần “strangle” monolith, chuyển features hiện có thành microservice từng phần.
4. Sau N năm, monolith biến mất.

Pattern này safe, gradual, không big-bang rewrite.

Sai lầm thường gặp

Sai lầm 1: Microservices vì hype

Đã nói nhiều. Resume-driven architecture.

Sai lầm 2: Distributed monolith

Tách thành microservices mà coupling chặt. Đã nói ở Bài 3.4.

Sai lầm 3: Quên 8 fallacies

Code distributed như local. Result: bug bí ẩn trong production.

Sai lầm 4: Không có observability từ ngày 1

Build distributed mà không có distributed tracing/centralized logging. Debug trở thành mò mẫm.

Sai lầm 5: Premature optimization

“We’ll need microservices in 5 years, let’s start now.” Sai. 80% project chết trước khi cần microservices. Start monolith, migrate when needed.

Tóm tắt

- **Monolithic default.** Distributed chỉ khi có lý do mạnh.
- **Cost của distributed:** latency, partial failure, consistency, ops, testing, security.
- **8 fallacies** của distributed computing, đọc kỹ.
- **Modular monolith** là middle ground tốt.
- **Strangler Fig** để migrate monolith → distributed an toàn.

Bài tiếp: [Layered Architecture](#), style phổ biến nhất historical.

5.3 Layered Architecture

Tóm tắt một dòng: Hệ chia thành tầng theo technical concern (UI, Business, Data). Đơn giản, được hiểu rộng, nhưng dễ rơi vào “sinkhole anti-pattern” và khó scale theo domain.

Nếu bạn đến từ Data Science

Layered Architecture có thể map sang model serving khá tự nhiên. Presentation layer là HTTP API hoặc batch endpoint. Business layer là inference logic: validate request, lấy feature, gọi model, format result. Persistence layer là nơi đọc model artifact, feature store, metadata database hoặc object storage.

Điểm cần tránh là để API layer biết quá nhiều chi tiết model và storage. Nếu endpoint trực tiếp load file pickle, query warehouse, transform feature và return response, bạn đang trộn layer. Khi muốn đổi model registry hoặc feature store, API sẽ bị sửa nhiều.

Topology

Quy tắc cốt lõi: **dependency flow downward**. Tầng trên gọi tầng dưới, không bao giờ ngược lại.

Closed vs Open layers

Closed layer (default)

Request phải đi qua *mọi* tầng. Presentation không skip Business để gọi Persistence trực tiếp.

Pros: enforce layering, encapsulation.

Cons: boilerplate khi tầng giữa chỉ “pass-through”.

Open layer

Một layer có thể be “open”, tầng trên có thể skip nó.

Vd: introduce Shared Services layer mà Presentation có thể skip:

Pros: avoid pass-through for simple ops.

Cons: phá thuần khiết. Phải document rõ layer nào open.

Heuristic: default closed. Open layer chỉ khi pass-through pain rõ.

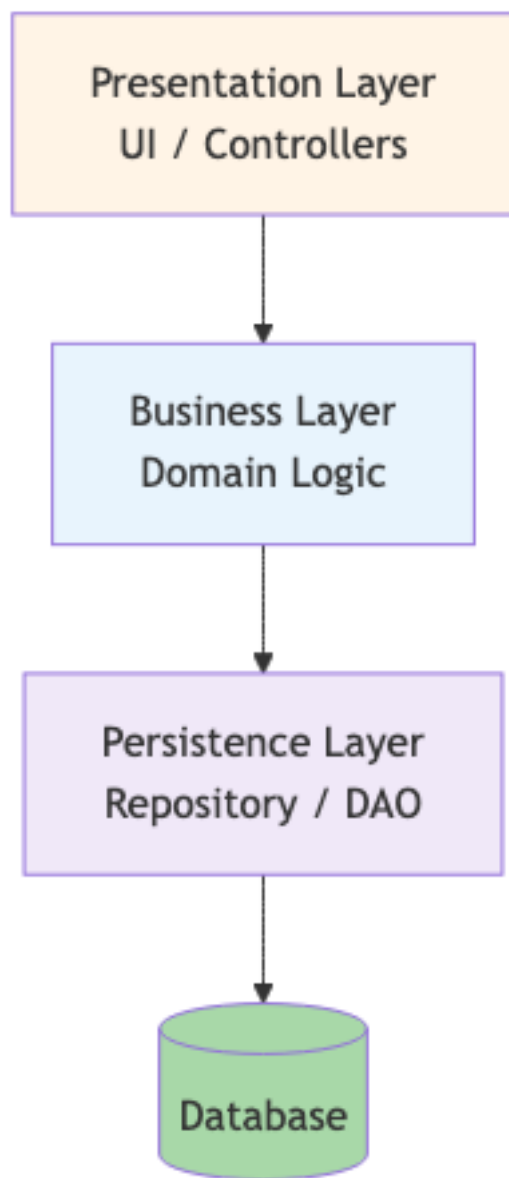
Sinkhole Anti-pattern

Khi 80%+ requests chỉ “pass through” các layer mà không có logic:

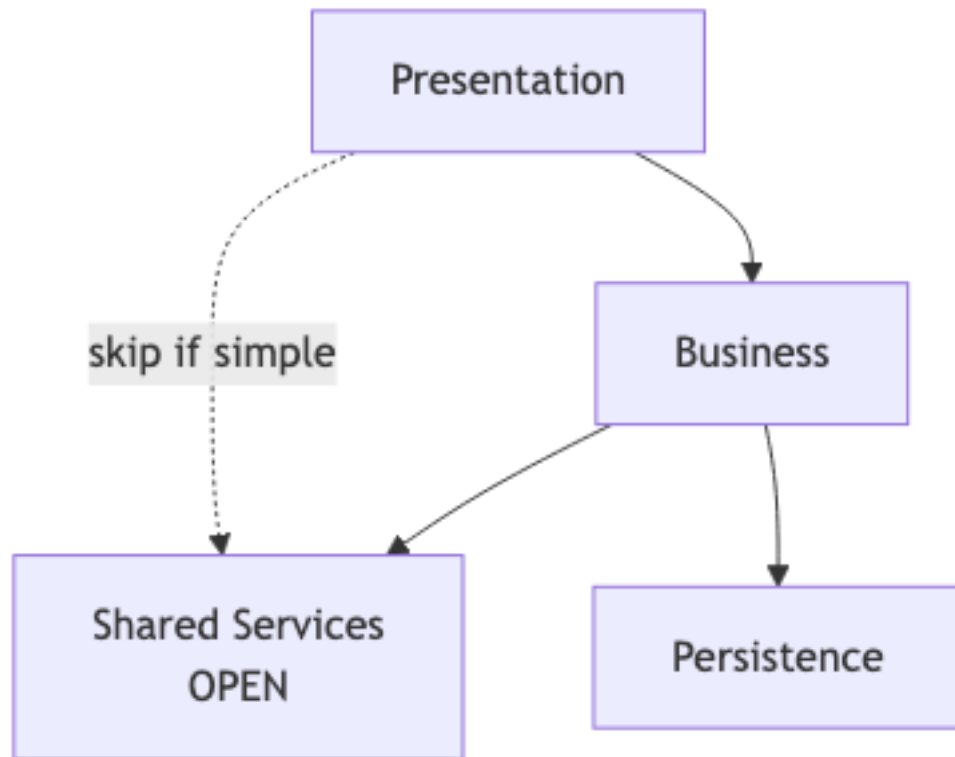
```
# Controller
def get_user(id):
    return user_service.get_user(id)

# Service
class UserService:
    def get_user(self, id):
        return self.repo.find_by_id(id)

# Repo
class UserRepo:
    def find_by_id(self, id):
        return db.query("SELECT * FROM users WHERE id=?", id)
```



Hình 3: Sơ đồ 14



Hình 4: Sơ đồ 15

3 tầng nhưng không tầng nào thực sự *thêm logic*. Đây là sinkhole, wasted abstraction.

Fix:

- **Option A:** Allow open layers cho pass-through cases.
- **Option B:** Drop layering cho read-heavy cases, use CQRS, write qua layered, read direct query.
- **Option C:** Reconsider style. Có thể layered không fit; xem service-based hoặc microkernel.

Khi dùng Layered

☐ Phù hợp khi:

- Hệ nhỏ-trung (< 50k LOC).
- Team < 15 people.
- Domain đơn giản (CRUD nặng).
- Cần onboard nhanh, layered được dạy phổ biến.
- Cần consistency với codebase legacy.

☐ Không phù hợp khi:

- Hệ phức tạp với nhiều domain (vertical slicing tốt hơn).
- Cần high scalability per-domain.
- Cần tách team theo domain (Conway's law conflict).
- Performance critical với many sinkholes.

Trade-off với QA

QA	Score	Note
Simplicity	□□□□□	Cấu trúc dễ hiểu nhất
Cost	□□□□□	1 deploy, 1 DB
Testability	□□□	OK nếu mỗi layer test với mock
Modularity (technical)	□□□	Layered theo tech concern
Modularity (domain)	□	Tệ, feature đung mọi layer
Scalability	□	Cannot scale parts separately
Deployability	□□	1 deploy = mọi thay đổi
Performance	□□□□	In-process call fast
Fault tolerance	□□	Single point of failure

Layered tối ưu cho **simplicity + cost**, hy sinh **scalability + modularity domain**.

Implementation trong code

Java/Spring example

```
// presentation/UserController.java
@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService service;

    @GetMapping("/{id}")
    public UserDto get(@PathVariable Long id) {
        return service.getUser(id);
    }
}

// business/UserService.java
@Service
public class UserService {
    private final UserRepository repo;

    public UserDto getUser(Long id) {
        User user = repo.findById(id).orElseThrow();
        return UserDto.from(user); // mapping
    }

    public UserDto createUser(CreateUserRequest req) {
        // validation, business rule
        User user = new User(req.email(), req.name());
        user.validate();
        return UserDto.from(repo.save(user));
    }
}
```

```
// persistence/UserRepository.java
@Repository
public interface UserRepository extends JpaRepository<User, Long> {}
```

Mỗi layer in own package. Spring auto-wire dependencies.

Python/Flask example

```
# presentation/user_blueprint.py
@app.route("/users/<int:id>")
def get_user(id):
    return jsonify(user_service.get(id))

# business/user_service.py
class UserService:
    def __init__(self, repo): self._repo = repo
    def get(self, id): return UserDto.from_entity(self._repo.find(id))

# persistence/user_repository.py
class UserRepository:
    def find(self, id):
        return User.query.filter_by(id=id).first()
```

Variants

Layered + Hexagonal

Combine layered với hexagonal (Cụm 2.7 DIP). Business layer định nghĩa abstraction; Persistence layer implement. Decouple business khỏi infrastructure.

Onion / Clean Architecture

Layered hơi twist: domain ở center, infrastructure ở ngoài. Dependency direction *vào trong* (DIP applied). Effectively layered with DIP enforced.

Cụm này không đi sâu, đọc *Clean Architecture* (Robert Martin) sau khoá.

Sai lầm thường gặp

Sai lầm 1: Skip layers ad-hoc

Code nhanh, controller gọi thẳng repository. Vài tháng sau, business layer chỉ là half-empty shell, không reliable để add logic.

Fix: discipline. Default closed; document open layer rõ ràng.

Sai lầm 2: Layered cho complex domain

Hệ có 20 sub-domain. Layered chỉ có 3 tầng, không match domain complexity. Mỗi feature touch nhiều file.

Fix: vertical slice (theo domain) + layered bên trong mỗi domain.

Sai lầm 3: Database-first design

Bắt đầu thiết kế từ database schema, work upward. Result: business layer là CRUD wrapper, không thực sự reflect domain.

Fix: design domain model first (business layer), DB schema follow. Hexagonal/Clean approach.

Sai lầm 4: Repository pattern abuse

Repository làm everything: SQL, caching, validation, business logic. Vi phạm SRP.

Fix: repository chỉ persistence. Cache trong service layer. Validation trong domain.

Tóm tắt

- **Layered**: tầng UI / Business / Persistence theo technical concern.
- **Closed default**, open chỉ khi pain rõ.
- **Sinkhole** = anti-pattern: pass-through layers.
- **Phù hợp**: hệ nhỏ-trung, domain đơn giản, team < 15.
- **Trade-off**: simplicity + cost tối ưu; scalability + domain modularity hy sinh.

Bài tiếp: [Pipeline Architecture](#), sequential transformation, Unix philosophy.

5.4 Pipeline Architecture

Tóm tắt một dòng: Pipeline Architecture tổ chức hệ thống như một dây chuyền xử lý: dữ liệu đi qua nhiều bước nhỏ, mỗi bước làm một việc rõ ràng, rồi chuyển output cho bước tiếp theo. Với Data Scientist, đây là cách nhìn kiến trúc hoá cho ETL, feature engineering, training pipeline, batch scoring và streaming inference.

Câu hỏi mở đầu

Hãy nhớ lại một notebook machine learning quen thuộc. Ban đầu notebook có thể rất gọn:

1. Load CSV.
2. Clean data.
3. Create features.
4. Train model.
5. Evaluate.
6. Save result.

Nhưng sau vài tuần, notebook bắt đầu dài ra: thêm xử lý missing value, thêm join với bảng mới, thêm encoding, thêm feature selection, thêm log metric, thêm save artifact, thêm export report. Cuối cùng bạn có một file rất dài, chạy được nhưng khó test, khó reuse và rất sợ sửa.

Pipeline Architecture là cách tách luồng xử lý đó thành nhiều bước nhỏ có boundary rõ. Thay vì một notebook làm tất cả, ta có nhiều filter, mỗi filter nhận input, xử lý đúng một trách nhiệm, rồi đưa output sang filter tiếp theo.

Topology cốt lõi



Hình 5: Sơ đồ 16

Hai thành phần quan trọng:

- **Filter:** đơn vị xử lý. Nó nhận input, biến đổi hoặc kiểm tra, rồi xuất output. Filter tốt nên nhỏ, dễ test và không cần biết filter trước/sau là ai.
- **Pipe:** kênh chuyển dữ liệu giữa filters. Pipe có thể là object trong memory, file, table, queue, Kafka topic, hoặc stream.

Một pipeline tốt giống dây chuyền sản xuất trong nhà máy. Mỗi công đoạn có nhiệm vụ rõ. Nếu sản phẩm lỗi ở công đoạn nào, bạn biết nên debug ở đâu. Nếu muốn thay công đoạn đóng gói, bạn không phải viết lại toàn bộ nhà máy.

Nếu bạn đến từ Data Science

Pipeline Architecture có lẽ là style tự nhiên nhất với bạn, vì phần lớn công việc DS vốn đã là pipeline. Điểm khác biệt là trong notebook, pipeline thường nằm ẩn trong thứ tự các cell. Trong production, pipeline cần được explicit hoá thành component, contract, schedule, monitoring và error handling.

Trong notebook	Trong architecture
Cell load data	Producer filter
Cell clean data	Transformer filter
Cell drop invalid rows	Tester filter

Trong notebook	Trong architecture
Cell feature engineering	Transformer filter
Cell train model	Consumer hoặc transformer đặc biệt
Cell save pickle	Sink filter
Biến dataframe truyền giữa cell	Pipe
Cell chạy lỗi	Pipeline failure cần retry/alert

Khi bạn biến notebook thành pipeline production, câu hỏi không chỉ là “code chạy không?”. Câu hỏi là:

- Mỗi step có input/output schema rõ không?
- Step nào có thể chạy lại an toàn nếu fail?
- Step nào mất nhiều thời gian nhất?
- Step nào chứa business logic quan trọng?
- Nếu data schema đổi, step nào detect được?
- Output của step này có thể reuse cho pipeline khác không?

Đó chính là tư duy kiến trúc.

Bốn loại filter

1. Producer

Producer tạo dữ liệu đầu vào cho pipeline. Nó có thể đọc từ file, API, database, data warehouse, Kafka topic, object storage hoặc sensor.

Ví dụ DS:

```
import pandas as pd

def load_transactions(path: str) -> pd.DataFrame:
    return pd.read_parquet(path)
```

Filter này không nên clean data, train model hoặc ghi output. Nó chỉ chịu trách nhiệm lấy dữ liệu vào pipeline.

2. Transformer

Transformer biến input thành output mới. Đây là loại filter phổ biến nhất trong data pipeline.

```
def add_recency_feature(df):
    df = df.copy()
    df["days_since_last_purchase"] = (
        df["snapshot_date"] - df["last_purchase_date"]
    ).dt.days
    return df
```

Transformer tốt nên deterministic: cùng input thì cho cùng output. Điều này giúp test và reproducibility dễ hơn.

3. Tester

Tester lọc hoặc validate input. Nó có thể drop records không hợp lệ, hoặc fail pipeline nếu schema sai.

```
def require_columns(df, required):
    missing = set(required) - set(df.columns)
    if missing:
        raise ValueError(f"Missing columns: {missing}")
    return df
```

Trong production ML, tester filter cực kỳ quan trọng. Nhiều model không fail khi data sai, chúng chỉ silently predict sai. Schema validation giúp lỗi xuất hiện sớm hơn.

4. Consumer hoặc sink

Consumer ghi kết quả ra đích cuối: database, warehouse, model registry, API response, dashboard, file, hoặc message topic.

```
def write_scores(df, table_name, warehouse_client):
    warehouse_client.write_table(df, table_name)
```

Sink là nơi pipeline tạo side effect. Vì vậy sink cần idempotency: nếu chạy lại cùng input, không được tạo duplicate hoặc corrupt data.

Unix Philosophy và vì sao nó vẫn đúng

Pipeline Architecture cụ thể hoá Unix Philosophy:

Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.

Ví dụ Unix pipeline:

```
cat access.log | grep "404" | awk '{print $7}' | sort | uniq -c | sort -rn | head -10
```

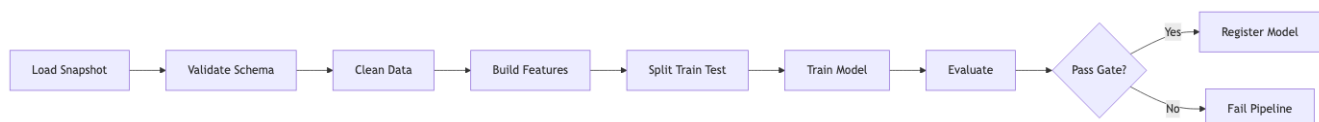
Pipeline này có vẻ đơn giản, nhưng chứa một bài học kiến trúc rất sâu:

- cat không biết grep sẽ làm gì.
- grep không biết awk sẽ làm gì.
- awk không biết kết quả cuối dùng để debug hay report.
- Các bước kết nối được vì chúng thống nhất format stream.

Trong ML/data system, “text stream” có thể được thay bằng DataFrame schema, Parquet schema, Avro schema, protobuf message hoặc table contract. Ý tưởng vẫn giống nhau: **filter độc lập, pipe có contract rõ**.

Ví dụ 1: Training pipeline

Một training pipeline production thường không nên là một notebook duy nhất. Nó có thể được tách như sau:



Hình 6: Sơ đồ 17

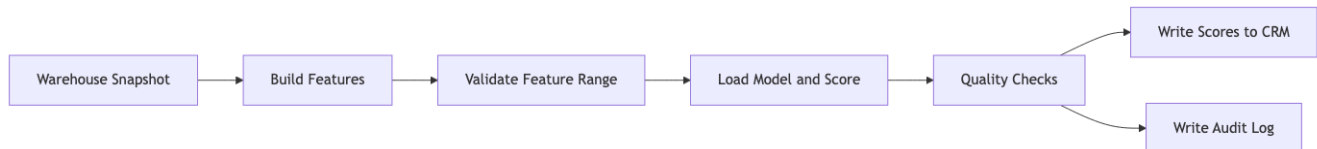
Điểm đáng chú ý là Evaluate không chỉ tính AUC. Nó có thể là quality gate:

- AUC không thấp hơn model hiện tại quá 1%.
- Bias metric không vượt threshold.
- Latency benchmark không vượt 100ms.
- Model artifact size không quá lớn.

Khi evaluate trở thành filter độc lập, bạn có thể test, version và thay đổi nó mà không sửa training code.

Ví dụ 2: Batch inference pipeline

Batch inference phù hợp khi business không cần phản hồi realtime. Ví dụ: mỗi đêm score churn cho toàn bộ customer rồi đẩy sang CRM.



Hình 7: Sơ đồ 18

Ưu điểm:

- Đơn giản hơn online inference.
- Dễ kiểm soát cost.
- Dễ audit vì mọi score được tạo theo batch version.
- Dễ replay nếu cần rebuild.

Nhược điểm:

- Score có thể stale.
- Không phù hợp use case cần phản ứng ngay, như fraud detection.

Ví dụ 3: Streaming feature pipeline

Khi freshness quan trọng, pipeline có thể chạy liên tục trên stream.



Hình 8: Sơ đồ 19

Ở đây pipe thường là Kafka topic hoặc stream internal của Flink/Spark Streaming. Trade-off bắt đầu phức tạp hơn:

- Throughput cao hơn batch.
- Freshness tốt hơn.
- Debug khó hơn.
- Operational cost cao hơn.
- Exactly-once hoặc at-least-once semantics cần được hiểu rõ.

Đừng chọn streaming chỉ vì nghe hiện đại. Hãy quay lại Cụm 4: business có thật sự cần freshness theo giây/phút không?

Cách implement pipeline

In-process pipeline

Dùng khi pipeline nhỏ, chạy trong một process, phù hợp notebook chuyển thành script hoặc batch job đơn giản.

```
from functools import reduce

def validate_schema(df):
    # validate columns and types
    return df

def clean_data(df):
    return df.dropna(subset=["customer_id"])

def build_features(df):
    df = df.copy()
    df["log_amount"] = (df["amount"] + 1).apply(np.log)
    return df

def score(df):
    df = df.copy()
    df["score"] = model.predict_proba(df[feature_cols])[:, 1]
    return df

filters = [validate_schema, clean_data, build_features, score]
result = reduce(lambda data, f: f(data), filters, input_df)
```

Ưu điểm: đơn giản, dễ hiểu, ít infrastructure. Nhược điểm: khó scale, monitoring hạn chế, failure recovery phải tự làm.

Orchestrated batch pipeline

Dùng Airflow, Dagster, Prefect, Luigi hoặc workflow engine tương tự.

```
extract >> validate >> build_features >> train >> evaluate >> register_model
```

Ở mức architecture, orchestrator không phải business logic. Nó điều phối dependency, schedule, retry, logging và visibility. Business logic vẫn nên nằm trong filter code riêng, để có thể test ngoài orchestrator.

Distributed streaming pipeline

Dùng Kafka Streams, Flink, Spark Structured Streaming hoặc cloud service tương tự.

```
Kafka topic raw-events
-> parser job
-> topic parsed-events
-> feature aggregation job
-> topic online-features
-> feature store sink
```

Dùng khi throughput hoặc freshness thật sự yêu cầu. Cái giá là bạn phải hiểu partitioning, event time, late events, retry, duplicate, schema evolution, monitoring lag.

Trade-off với Quality Attributes

QA	Pipeline hỗ trợ tốt?	Lý do
Modularity	Rất tốt	Mỗi filter có trách nhiệm rõ
Testability	Rất tốt	Test từng filter độc lập
Reusability	Tốt	Filter có thể dùng lại ở pipeline khác
Observability	Tốt nếu instrument đúng	Có thể đo latency/failure từng step
Throughput	Tốt	Có thể parallelize filter hoặc partition data
Latency	Trung bình	Latency tổng bằng tổng nhiều step
Simplicity	Tốt với pipeline nhỏ	Nhưng giảm khi branching/state nhiều
Consistency	Tùy implementation	Distributed pipeline dễ gặp duplicate/out-of-order
Debuggability	Tốt nếu có checkpoint	Khó nếu pipe không lưu intermediate output

Khi nào dùng Pipeline Architecture?

Phù hợp

- ETL/ELT pipeline.
- Feature engineering pipeline.
- Training pipeline.
- Batch inference.
- Log processing.
- Image/video processing.
- Compiler/build system.
- Streaming aggregation khi data flow rõ.

Không phù hợp

- Workflow có business transaction phức tạp.
- User-facing interaction cần nhiều decision branch.
- Domain logic cần shared state dày đặc.
- Flow mà step sau cần gọi ngược step trước liên tục.

- Hệ cần orchestration phức tạp kiểu saga, compensation, approval.

Một dấu hiệu sai style: bạn phải nhét quá nhiều `if/else`, `loop`, `rollback`, `shared database update` vào từng filter. Khi đó có thể bạn đang cố ép một business workflow thành data pipeline.

Design rules cho pipeline tốt

Rule 1: Contract giữa filters phải rõ

Filter A output gì, Filter B expect gì? Trong DS, đây thường là DataFrame schema hoặc table schema. Nếu contract mơ hồ, pipeline sẽ lỗi ở runtime.

Nên có:

- Required columns.
- Data types.
- Nullability.
- Value ranges.
- Time window.
- Semantic meaning.

Rule 2: Filter càng stateless càng tốt

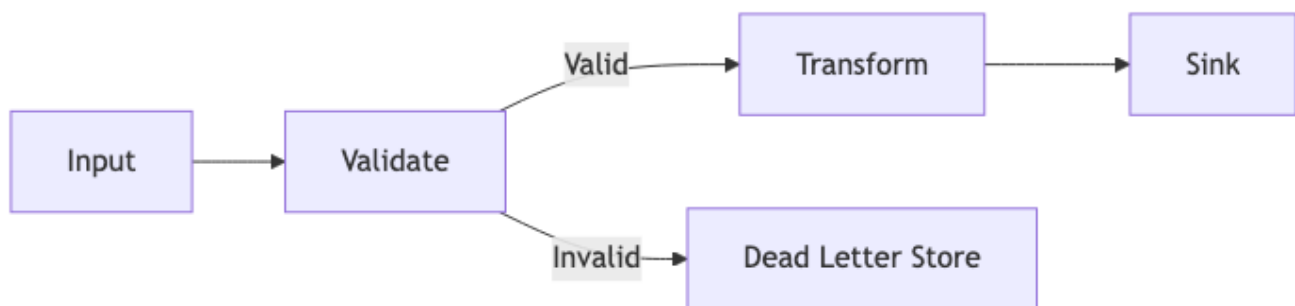
Stateless filter dễ parallel, replay và test. Nếu cần state, hãy đặt state ở nơi rõ ràng như feature store, database, checkpoint store hoặc model registry.

Rule 3: Side effect phải nằm ở sink rõ ràng

Một filter vừa transform data vừa ghi database vừa gửi email là filter nguy hiểm. Nó khó retry và khó test. Hãy cố giữ transform thuần, side effect ở sink.

Rule 4: Error path cũng là một pipeline

Production pipeline không chỉ có happy path. Cần thiết kế error path:



Hình 9: Sơ đồ 20

Dead letter store giúp bạn inspect record lỗi, sửa data hoặc replay sau.

Rule 5: Mỗi step cần metric riêng

Đừng chỉ monitor pipeline success/failure. Hãy monitor từng filter:

- Input count.
- Output count.
- Error count.
- Processing latency.
- Data quality metric.
- Feature null rate.
- Schema drift.

Với ML, nhiều lỗi không làm pipeline fail. Chúng làm model âm thầm tệ đi. Monitoring từng step giúp phát hiện sớm.

Sai lầm thường gặp

Sai lầm 1: Notebook là pipeline production

Notebook tốt cho exploration, không tốt làm production boundary. Cell order có thể không rõ, state ẩn nhiều, khó test tự động, khó schedule và khó review.

Fix: tách logic thành module Python, dùng notebook cho exploration/report, dùng orchestrator cho production.

Sai lầm 2: God filter

Một function `process_data()` làm validate, clean, join, feature, train, save. Tên nghe như pipeline, nhưng thật ra là monolith trong một function.

Fix: tách theo trách nhiệm và đặt tên filter cụ thể: `validate_schema`, `remove_outliers`, `build_recency_features`, `train_xgboost`, `register_model`.

Sai lầm 3: Không lưu intermediate output

Pipeline fail ở cuối, nhưng bạn không biết input của step cuối là gì. Debug phải chạy lại từ đầu rất tốn thời gian.

Fix: checkpoint ở boundary quan trọng. Với batch pipeline, có thể lưu Parquet sau validation/feature. Với streaming, có topic trung gian hoặc state store.

Sai lầm 4: Không version schema và model

Feature pipeline thay đổi nhưng model serving vẫn expect schema cũ. Kết quả là runtime error hoặc prediction sai.

Fix: version data contract, feature schema, model artifact. Model registry nên lưu cả feature schema và dependency version.

Sai lầm 5: Chọn streaming khi batch đủ

Streaming pipeline rất hấp dẫn, nhưng khó vận hành hơn batch nhiều. Nếu business chỉ cần report mỗi sáng, Airflow batch có thể tốt hơn Kafka/Flink.

Fix: bắt đầu từ QA. Nếu freshness target là 24 giờ, batch là ứng viên mạnh. Nếu target là 5 giây, streaming mới đáng cân nhắc.

Self-check

1. Một notebook ML gần đây của bạn có thể tách thành những filters nào?
2. Pipe giữa các filters là gì: DataFrame, file, table, queue hay topic?
3. Filter nào có side effect? Side effect đó có idempotent không?
4. Nếu schema input đổi, pipeline phát hiện ở step nào?
5. Nếu step scoring fail, bạn có replay được từ feature checkpoint không?

Tóm tắt

- **Pipeline Architecture** tổ chức hệ thành filters và pipes.
- Với Data Scientist, đây là cách production hoá ETL, feature engineering, training và scoring.
- Filter tốt nên nhỏ, stateless, testable, có input/output contract rõ.
- Pipe có thể là memory object, file, table, queue, Kafka topic hoặc stream.
- Pipeline tối ưu modularity, testability, reusability và throughput.
- Pipeline không phù hợp cho workflow business phức tạp, nhiều transaction, nhiều rollback.
- Đừng chọn streaming nếu batch đã đủ đáp ứng Quality Attributes.

Bài tiếp: [Microkernel Architecture](#), cách thiết kế core + plug-in, rất hữu ích khi bạn muốn thêm model algorithm, metric hoặc feature transformation mà không sửa core.

5.5 Microkernel Architecture

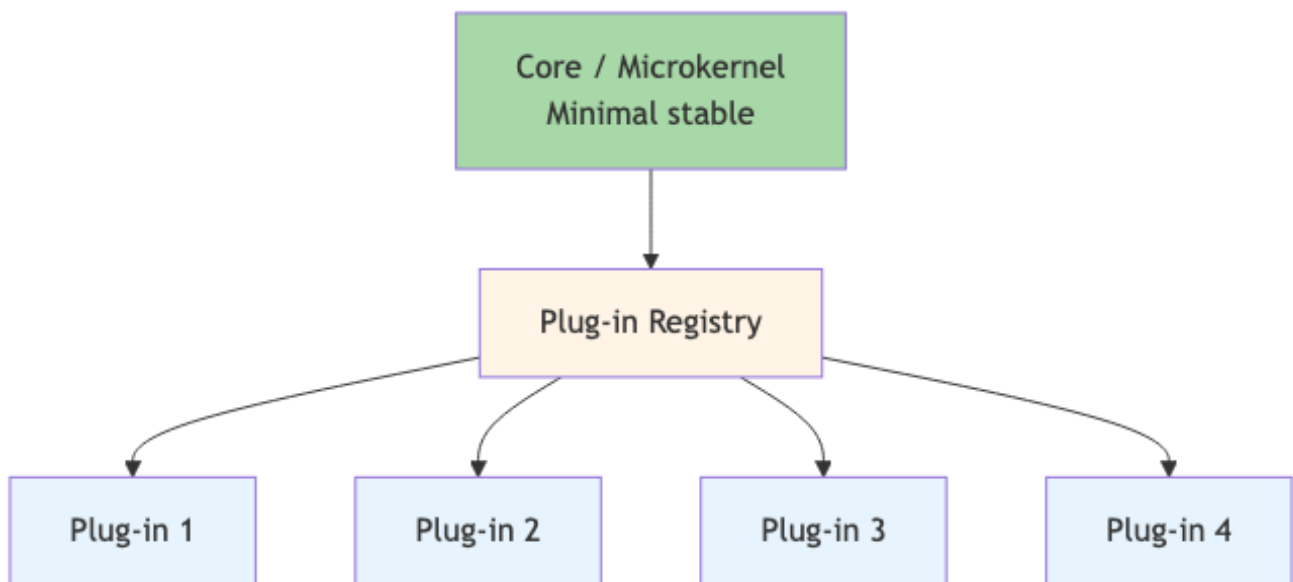
Tóm tắt một dòng: Hệ thống chia thành một core stable nhỏ + nhiều plug-in mở rộng tính năng. OCP ở scale lớn. Phù hợp cho product cần third-party extension hoặc customization mạnh.

Nếu bạn đến từ Data Science

Microkernel rất hợp với platform cho nhiều model hoặc nhiều thuật toán. Core giữ những thứ ổn định: data contract, pipeline orchestration, registry, logging, permission. Plug-in là phần thay đổi nhiều: model algorithm, feature transformation, evaluation metric, explanation method.

Ví dụ một evaluation platform có core đọc prediction/label và tính report chuẩn. Team có thể thêm metric plug-in như AUC, F1, calibration error, fairness metric mà không sửa core. Đây là OCP ở mức architecture.

Topology cốt lõi



Hình 10: Sơ đồ 21

Ba thành phần:

- **Core (kernel):** minimal, stable. Chứa logic *chung nhất* (vd: window management, plug-in lifecycle, common utilities).
- **Plug-in:** standalone module. Implement một feature cụ thể. Communicate với core qua well-defined contract.
- **Registry:** nơi core biết về plug-in nào available. Discovery cơ chế: file system scan, manifest config, hoặc remote registry.

Quy tắc cốt lõi: **core không biết về plug-in cụ thể**. Plug-in implement interface mà core defined. Add/remove plug-in không touch core code.

Lợi ích vs Layered

So với layered:

- **Extensibility:** layered cần modify code để add feature. Microkernel chỉ cần drop in plug-in.
- **Customization:** user/admin có thể enable/disable plug-in. Layered có toàn bộ feature on-or-off.
- **Third-party ecosystem:** plug-in API mở cho dev khác. Eg: VS Code extension marketplace.
- **Independent versioning:** core và plug-in version độc lập.

Trade-off với layered:

- **Complexity:** phải design plug-in API cẩn thận, manage lifecycle (load, init, dispose).
- **Performance:** cross-plug-in call có overhead (interface dispatch, marshal data).
- **Security:** plug-in third-party có thể malicious, cần sandboxing.

Examples thực tế

VS Code

- **Core:** window manager, file system access, editor core, plug-in lifecycle.
- **Plug-in:** hàng ngàn extension cho language, theme, debugging, AI assistance, source control.
- **API:** TypeScript SDK với contracts (commands, providers, language services).

Eclipse IDE

Tiền bối của VS Code. Core (Equinox OSGi container) + bundles (plug-ins). Mọi feature kể cả Java editor là plug-in.

Chrome / Firefox

- **Core:** Chromium / Gecko engine.
- **Plug-in:** extension cho ad-blocker, password manager, dev tools.

WordPress

- **Core:** post system, user management, admin UI, plug-in lifecycle.
- **Plug-in:** SEO, e-commerce (WooCommerce), gallery, forms, ... > 50,000 plug-in.

CMS / E-commerce platforms

Magento, Shopify, Drupal đều microkernel. Merchant install plug-in để add feature mà không cần touch core.

DAW (Digital Audio Workstation)

Pro Tools, Ableton: core audio engine + plug-in synthesizer/effect (VST3, AU).

Plug-in Contract Design

Đây là phần khó nhất. Contract = interface mà mọi plug-in phải implement. Thiết kế kém □ plug-in ecosystem chết.

Loại 1: Event/Hook system

Core emit event. Plug-in subscribe.

```
# Core
class EventBus:
    def emit(self, event_name, payload):
        for handler in self._handlers.get(event_name, []):
            handler(payload)

# Plug-in
def on_user_login(payload):
    log.info(f"User {payload.user_id} logged in")

core.event_bus.subscribe("user.login", on_user_login)
```

Vd: WordPress hooks (add_action, add_filter).

Pros: loose coupling. Plug-in không block core.

Cons: hard to reason about order; debugging khó.

Loại 2: Service registration

Plug-in register service vào registry. Core query.

```
# Plug-in
class MyAuthProvider(AuthProvider):
    def authenticate(self, creds): ...

core.registry.register("auth", "oauth", MyAuthProvider())

# Core
auth = core.registry.get("auth", "oauth")
user = auth.authenticate(creds)
```

Vd: Spring beans, Java ServiceLoader, .NET DI container.

Pros: explicit, type-safe.

Cons: plug-in phải khai báo trước khi dùng.

Loại 3: Command pattern

Core có command registry. Plug-in register commands.

```
// VS Code extension
context.subscriptions.push(
    vscode.commands.registerCommand('myExt.helloWorld', () => {
        vscode.window.showInformationMessage('Hello!');
    })
);
```

Pros: discoverability (user thấy commands trong palette).

Cons: chỉ cho command-style interaction, không cho data flow phức tạp.

Loại 4: Contribution points (Eclipse model)

Core declare contribution point. Plug-in declare what it contributes.

```
<!-- Plug-in's plugin.xml -->
<extension point="org.eclipse.ui.editors">
  <editor id="my.editor" class="my.EditorImpl"/>
</extension>
```

Core scan all plug-ins for contributions, wire up.

Pros: declarative, lazy loading.

Cons: XML configuration verbose; learning curve cao.

Plug-in Lifecycle

Plug-in cần được manage:

1. **Discovery:** core scan plug-in folder, registry, hoặc remote source.
2. **Load:** load code (DLL, JAR, JS bundle) vào process.
3. **Initialize:** gọi plug-in's `init()`; plug-in register services/commands.
4. **Activate:** plug-in start using core services. Lazy activation = activate khi user thật sự dùng.
5. **Deactivate:** clean up resources, unsubscribe events.
6. **Unload:** remove code khỏi memory.

Mỗi step có error handling. Bad plug-in không được crash core.

Sandboxing (cho third-party plug-ins)

Plug-in third-party có thể malicious hoặc bug. Cần isolation:

Process isolation

Plug-in run trong child process. Crash plug-in không vỡ core. IPC qua pipe/socket.

Vd: Chrome render extension trong child process.

Cost: IPC slow hơn in-process call (~100x).

Sandboxed runtime

Plug-in run trong VM với limited permission. Vd: V8 isolate, Wasm sandbox.

Cost: limited capability (no syscalls, limited memory).

Permission system

Plug-in declare permission cần (network, file system, mic, camera). User approve khi install.

Vd: Browser extension permissions, Android app permissions.

Trade-off với QA

QA	Score	Note
Extensibility	□□□□□	Best in class

QA	Score	Note
Modularity	□□□□	Core + plug-in boundary cứng
Performance	□□□	Plug-in call có overhead
Maintainability (core)	□□□□	Core nhỏ, stable, dễ maintain
Maintainability (plug-in)	□□	Plug-in compatibility issue khi core update
Testability	□□□	Test plug-in dễ; test core+plug-in interactions khó
Deployability	□□□□	Plug-in deploy/update độc lập
Complexity	□□	Plug-in API design + lifecycle phức tạp

Microkernel tối ưu cho **extensibility**, sacrificing performance + simplicity.

Khi dùng Microkernel

□ Phù hợp:

- Product cần customization mạnh (CMS, e-commerce, IDE).
- Có ecosystem third-party (extension marketplace).
- Core feature ổn định, variation ở edge cases.
- Cần independent versioning core vs feature.

□ Không phù hợp:

- App nội bộ với fixed feature set.
- Performance critical (real-time, low-latency).
- Team không có resource để maintain plug-in API.

Variants

Microkernel + Distributed plug-ins

Plug-in chạy ở remote service. Core gọi qua HTTP/gRPC.

Vd: cloud services with extension hooks (Salesforce, Zendesk).

Cost: thêm distributed cost (Bài 5.2).

Microkernel + Module Federation (Frontend)

Webpack 5 Module Federation: frontend microkernel với React modules từ multiple repos.

```
// host (microkernel)
import('plugin/Feature').then(module => {
  module.default.mount('#container');
});
```

Plug-in code loaded lazily từ CDN. Independent deployment cho frontend teams.

Sai lầm thường gặp

Sai lầm 1: Core quá lớn

Core “nhỏ” theo plan nhưng dần phình ra với utilities, common features. Plug-in API không thực sự minimal.

Fix: discipline. Mọi feature mới hỏi “đây có phải core không?”. Default: not core, làm plug-in.

Sai lầm 2: API không backward compatible

Core release mới phá plug-in cũ. Ecosystem chết.

Fix: SemVer cho plug-in API. Major version chỉ khi *force break*. Deprecate API trước khi remove.

Sai lầm 3: Plug-in coupling lẫn nhau

Plug-in A depend Plug-in B. Khi cài A không có B ☐ vỡ.

Fix: plug-in chỉ depend Core API. Coordination giữa plug-in qua event bus của core, không direct call.

Sai lầm 4: Performance overhead bị bỏ qua

Plug-in call qua reflection/dispatcher. Slow trong hot path.

Fix: profile hot path. Cache plug-in instance. Pre-compile plug-in dispatch.

Sai lầm 5: Security model yếu

Plug-in có full access vào core internal. Một bad plug-in crash hoặc data theft.

Fix: define permission model. Sandbox plug-in. Code-review marketplace.

Tóm tắt

- **Microkernel**: core stable + plug-ins extensible.
- **OCP ở scale lớn**: core closed for modification, open for extension via plug-in.
- **Plug-in contract**: event/hook, service registration, command, contribution point.
- **Lifecycle + sandboxing** quan trọng cho production.
- **Use cases**: IDE, browser, CMS, e-commerce, DAW.
- **Trade-off**: extensibility tối ưu, sacrificing performance + complexity.

Tổng kết Cụm 5

Hết Cụm 5. Tóm tắt 3 styles:

Style	Tối ưu	Sacrificing	Use case điển hình
Layered	Simplicity, cost	Scalability, domain modularity	CRUD app, internal tools
Pipeline	Composability, reusability	Stateful logic, branching	ETL, compiler, image processing
Microkernel	Extensibility	Performance, complexity	IDE, CMS, browser

Plus **Monolithic vs Distributed** baseline (Bài 5.2).

Cụm tiếp: [Cụm 6 - Distributed Styles](#), bốn style phân tán quan trọng nhất.